

Dokumentace ROS2 projektu pro autonomní průjezd koridorem

BPC-PRP tým
Vysoké učení technické v Brně
Fakulta informačních technologií
Brno, Česká republika

Abstract—Tento dokument popisuje implementaci projektu `prp_project` postaveného nad ROS2. Řešení kombinuje zpracování dat z LiDARu, IMU a kamery do modulární architektury uzlů, které jsou propojeny přes standardní ROS topiky. Jádrem systému tvoří stavový automat `CorridorLoop`, jenž realizuje autonomní pohyb robota koridorem, detekci křížovatek, rozhodování o směru jízdy a přepínání mezi dvěma řídicími režimy: orientace podle IMU a centrování podle LiDARu. Dokument shrnuje strukturu kódu, hlavní algoritmy, datové toky mezi uzly a praktické volby tuningových konstant pro průjezd bludištěm.

Index Terms—ROS2, mobilní robotika, PID regulace, LiDAR, IMU, ArUco

I. INTRODUCTION

Projekt `prp_project` řeší autonomní řízení malého diferenciálního robota v laboratorním prostředí. Cílem je stabilní průjezd koridory a bludištěm, detekce křížovatek a reakce na instrukce z vizuálních markerů. Implementace je postavena nad ROS2 (`roscpp`) a je rozdělena do více uzlů podle typu senzoru a řídicí logiky. Tento modulární přístup zjednodušuje ladění, testování i budoucí rozšíření.

Z hlediska architektury jsou důležité tři vrstvy. První vrstva je senzorická: uzly `LidarNode`, `ImuNode` a `CameraNode` převádí surová data robota na interpretovatelné veličiny (sektorové vzdálenosti, yaw, ArUco instrukce). Druhá vrstva je řídicí: `CorridorLoop` periodicky počítá akce pro motory. Třetí vrstva je akční, kde se publikováním do topiku `/bpc_prp_robot/set_motor_speeds` odesílají povely přímo na podvozek.

Vstupní bod aplikace (`main.cpp`) vytváří šest samostatných executorů a pro každý uzel spouští vlastní vlákno. Díky tomu se navzájem neblokují dlouhé callbacky a zachovává se předvídatelná perioda řídicích smyček. Současně je tím odděleno zpracování kamery a LiDARu od řídicích rozhodnutí, která jsou časově citlivá.

Pozornost byla věnována i robustnosti. V kódu jsou dostupné sanity kontroly časového kroku, filtrace neplatných měření, hystereze při přepínání řídicích režimů a timeouty otáčení. Použitá regulace navazuje na klasické principy P/PID řízení a na robotické postupy práce s lokálním vnímáním prostředí. Cílem nebylo vytvořit globální mapování, ale spolehlivé lokální chování v podmínkách zadání.

II. ARCHITEKTURA A IMPLEMENTACE

A. Senzorické uzly

`LidarNode` odebírá `/bpc_prp_robot/lidar` a každý scan rozděljuje do osmi sektorů (N, NE, E, SE, S, SW, W, NW). Pro každý sektor se počítá medián vzdálenosti, což je odolnější vůči náhodným špičkám než aritmetický průměr. Výsledkem je osm topiků `/lidar_node/*`, které jsou snadno použitelné v dalších řídicích uzlech.

`ImuNode` přijímá zprávy `sensor_msgs/Imu`, v úvodu provede kalibraci offsetu gyroskopu a následně integruje úhlovou rychlost kolem osy Z na yaw. Hodnota je publikována do `/imu_node/yaw`. Implementace obsahuje ochranu před nestandardním dt, aby nedošlo k jednorázovým skokům odhadu.

`CameraNode` dekóduje komprimovaný obraz, detekuje ArUco markery a publikováním instrukcí 0/1/2 do `/corridor_loop/instruction` předává požadovaný směr jízdy (rovně/vlevo/vpravo). Zároveň publikuje anotovaný obraz pro diagnostiku.

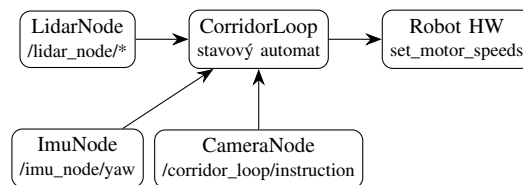


Figure 1. Blokové schéma komunikace hlavních ROS2 uzlů.

B. Řídicí smyčka pro koridor

Nejkomplexnější část projektu je `CorridorLoop`. Jde o stavový automat se stavy `CALIBRATING`, `ALIGNING`, `DRIVE_FORWARD`, `STOP_AND_SCAN` a trojicí stavů otáčení. Aktivace probíhá tlačítkem robota; po zapnutí se čekají validní data IMU a systém se zarovná do nejslibnějšího směru.

1) *Detail použití IMU*: IMU se používá jako primární zdroj orientace. V uzlu `ImuNode` je po startu provedena kalibrace gyroskopu: po omezenou dobu se sbírají jen vzorky s malou absolutní hodnotou `gyro_z` a z nich se vypočte offset. Teprve poté se začne integrovat yaw. Tím se výrazně omezí drift způsobený konstantní chybou senzoru.

Pro každou zprávu v běhovém režimu platí:

$$\omega_z^{corr} = \omega_z - \omega_{offset}, \quad \psi_{k+1} = \text{wrap}(\psi_k + \omega_z^{corr} \Delta t),$$

kde funkce `wrap` převádí úhel do intervalu $(-\pi, \pi]$. Uzel zároveň filtruje nevalidní časový krok ($\Delta t \leq 0$ nebo příliš velké Δt), aby neproběhla chybná integrace po výpadku.

V `CorridorLoop` se yaw používá ve dvou místech:

- při `DRIVE_FORWARD` pro držení směru vůči `heading_ref`,
- při `TURN_*` pro přesné dokončení otočení na cílový úhel.

Chyba orientace je $e_h = \text{wrap}(\psi_{ref} - \psi)$ a regulační zásah vzniká přes PID. Výstup se následně mapuje na diferenciální řízení kol:

$$v_L = v_{base} - u, \quad v_R = v_{base} + u.$$

2) *Detail použití LiDARu*: LiDAR se používá pro lokální geometrii prostředí. `LidarNode` převádí 2D scan do osmi sektorů a publikuje pro každý sektor medián vzdálenosti. Pro centrování v koridoru je klíčový rozdíl diagonálních sektorů NW a NE:

$$e_c = d_{NW} - d_{NE}.$$

Pokud je $e_c > 0$, robot je blíže pravé stěně a korekce směřuje vlevo; pokud $e_c < 0$, naopak. Korekce je omezená saturací, aby nedocházelo k agresivním oscilacím při šumu.

LiDAR dále slouží k rozhodování stavového automatu:

- **detekce čelní zdi**: podle sektoru N a prahu `STOP_THRESHOLD`,
- **detekce křižovatky**: podle počtu otevřených směrů (N/E/W/S),
- **zarovnání po otočení**: kombinací úhlové tolerance a podobnosti NW/NE.

Přepnutí mezi IMU a LiDAR režimem je asymetrické: vstup do LiDAR režimu vyžaduje validní W, NW, E i NE, ale návrat do IMU nastává ihned při ztrátě jedné diagonály. Tento návrh je stabilní v úzkém koridoru a současně rychle reaguje při otevření prostoru.

Při nálezu zdi před robotem nebo při potvrzené křižovatce se přechází do `STOP_AND_SCAN`. Uzel vyhodnotí otevřenou směr, aplikuje poslední uloženou instrukci z kamery a rozhodne o manévru. Pokud je instrukce fyzicky neproveditelná (např. vlevo je zed'), použije se automatická strategie volící volnější stranu. Otáčení je řízeno P regulátorem nad úhlovou chybou a doplněno timeoutem, aby nemohlo dojít k zablokování stavového automatu.

III. CONCLUSION

Předložená implementace potvrzuje, že i na relativně jednoduchém hardware lze dosáhnout robustního autonomního chování při vhodně navržené modulární architektuře. Rozdělení na senzorické uzly a řídicí smyčky zlepšuje přehlednost kódu, umožňuje postupné ladění jednotlivých částí a usnadňuje náhradu konkrétního algoritmu bez zásahu do celého systému.

Nejvýznamnějším prvkem je `CorridorLoop`, který kombinuje stavové řízení s přepínáním mezi IMU a LiDAR regulací. Praktické prvky jako lock instrukcí z `ArUco`, hystereze při přechodu režimů, timeout otáčení nebo potlačení falešné detekce křižovatky po manévru se ukázaly jako klíčové pro

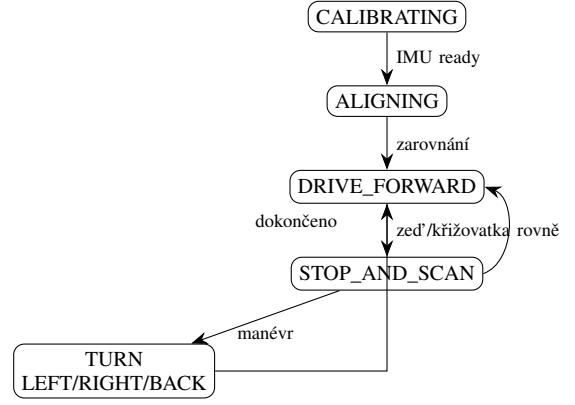


Figure 2. Zjednodušený stavový diagram řízení v uzlu `CorridorLoop`.

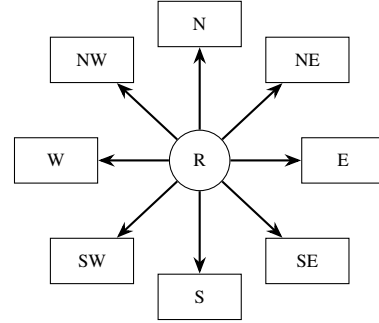


Figure 3. Sektorace LiDARu používaná v uzlu `LidarNode`.

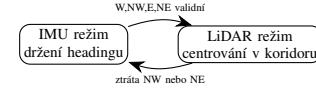


Figure 4. Asymetrické přepínání regulačních režimů v `DRIVE_FORWARD`.

stabilní provoz v reálném prostředí. Tyto mechanismy často rozhodují víc než samotná volba regulační rovnice.

Řešení je zaměřené čistě na průjezd bludištěm v koridorovém prostředí, kde se kombinuje lokální vnímání překážek, orientace podle IMU a rozhodování v křižovatkách podle aktuální geometrie prostoru a vizuálních instrukcí.

Za přirozené další kroky lze považovat: (i) formální vyhodnocení kvality řízení pomocí metrik překmitu, ustálení a odchylky od osy koridoru, (ii) fúzní odhad orientace (např. doplnění akcelerometru k gyru) a (iii) rozšířenou strategii plánování na sekvenci více instrukcí. Současná verze je nicméně funkční, konzistentní a připravená jako základ pro navazující experimenty.

ACKNOWLEDGMENT

Dokument vznikl v rámci předmětu BPC-PRP a vychází z implementace týmového semestrálního projektu.